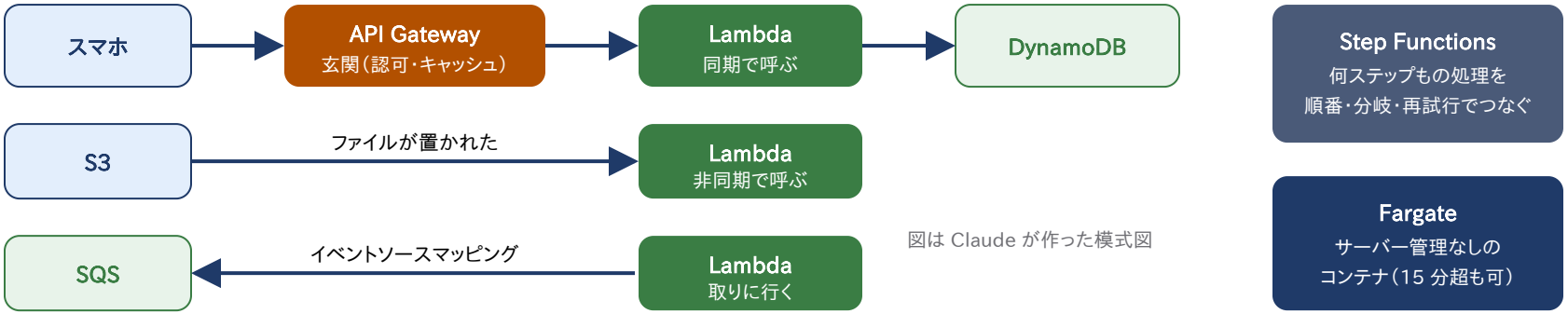
AWS SAA（SAA-C03）　分野2 弾力性に優れたアーキテクチャの設計　#07

サーバーレス

サーバーを持たずに動かす。Lambda の設定と呼び出し方、API Gateway、Step Functions、Fargate の使い分け

各ページの下に、根拠にした AWS の公式ドキュメント（Lambda・API Gateway・Step Functions・Amazon ECS）を書いた（青い文字を押すと開く）。「」の中はドキュメントの原文どおり（日本語版は機械翻訳のため、語順が不自然なところもそのまま）。会社名や構成の例は Claude が作った例

全体像 試験ガイド タスク 2.1



タスク 2.1 の知識・スキル（このPDFに関係する部分の要約）	このPDF
サーバーレステクノロジーとパターン（AWS Fargate、AWS Lambda など）	第1・2・4章
API の作成と管理（Amazon API Gateway、REST API など）	第3章
ワークフローオーケストレーション（AWS Step Functions など）	第4章
マイクロサービスの設計原則（ステートレスとステートフル）	第1・4章

出典：AWS「コンテンツ分野 2: 弾力性に優れたアーキテクチャの設計」タスク 2.1

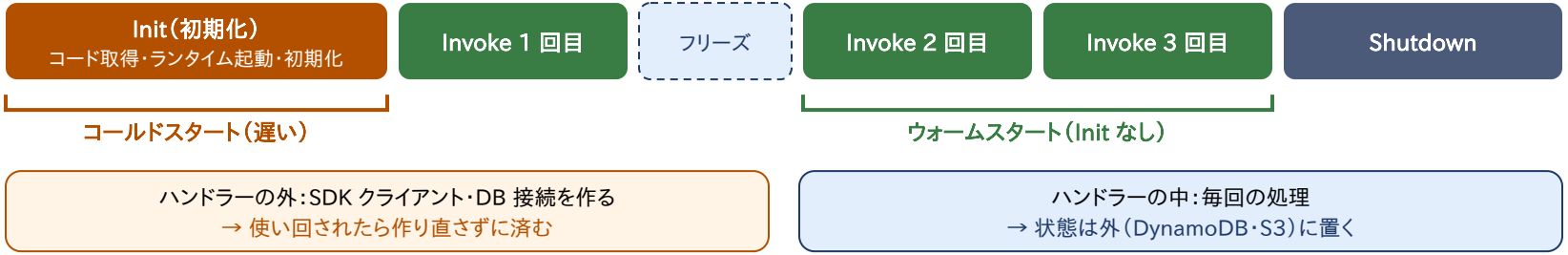
第1章

Lambda のしくみと設定

関数は実行環境の中で動く。メモリ・時間・ディスク・依存関係・つなぎ方を決める。

実行環境のライフサイクルとコールドスタート

1 つの実行環境の一生（Claude が作った模式図）



コールドスタートは少ない：AWS によると呼び出しの 1% 未満。それでも困るなら、プロビジョニングされた同時実行か SnapStart（第2章）。

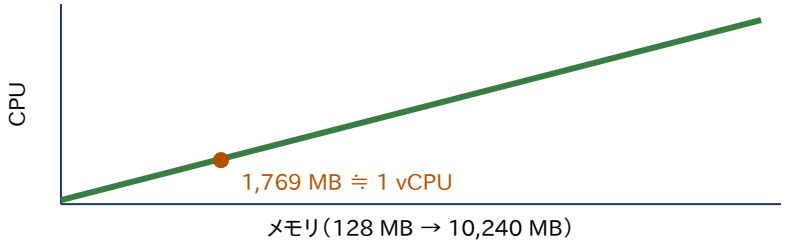
使い回しは保証されない：次の呼び出しが同じ環境に来るとは限らない。ログインの状態などは関数の中に持たない（ステートレス）。

出典：Lambda 実行環境のライフサイクルの概要（「Init フェーズは 10 秒に制限されています。」「通常、コールドスタートは呼び出しの 1% 未満で発生します。」）／AWS Lambda 関数を使用するためのベストプラクティス（「関数ハンドラー外で SDK クライアントとデータベース接続を初期化し、静的なアセットを /tmp ディレクトリにローカルにキャッシュします。」）

Lambda の数字:これだけは覚える

項目	値
メモリ	128 MB〜10,240 MB(1 MB 単位)
タイムアウト	デフォルト 3 秒・最大 **900 秒(15 分)**
/tmp	512 MB〜10,240 MB(一時・共有しない)
ペイロード	同期 6 MB／非同期 1 MB
.zip	圧縮 50 MB・解凍後 250 MB(レイヤー込み)
コンテナイメージ	10 GB(非圧縮)
レイヤー	1 関数に 5 つまで
環境変数	全部で 4 KB

CPU はメモリについてくる



CPU は直接決められない:CPU が足りない(遅い)ならメモリを増やす。vCPU 数の設定はない。

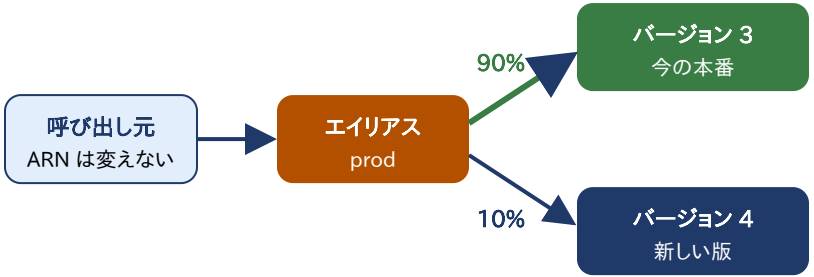
15 分を超える処理: Lambda は無理。Fargate・EC2・AWS Batch に移すか、小さく分けて Step Functions でつなぐ。

出典:Lambda クォータ／Lambda 関数のメモリを設定(「Lambda は、設定されたメモリの量に比例して CPU パワーを割り当てます。」)／Lambda 関数のタイムアウトを設定する(「この設定のデフォルト値は 3 秒ですが、最大値の 900 秒 (15 分) まで 1 秒単位で調整できます。」)

レイヤー・コンテナイメージ・バージョンとエイリアス

しくみ	何のため
レイヤー	共通ライブラリを .zip にまとめ、多くの関数で使い回す。コードと別に更新できる
コンテナイメージ	250 MB に入らない大きな依存関係（ECR に置く、最大 10 GB）。動きは Lambda のまま（15 分）
バージョン	発行した時点で固める（変更不可）。編集中は $LATEST
エイリアス	バージョンを指す名前。付け替え・2 つのバージョンへの振り分け
関数 URL	関数専用の HTTPS の入口。認証は IAM かなし。API キーやキャッシュは API Gateway

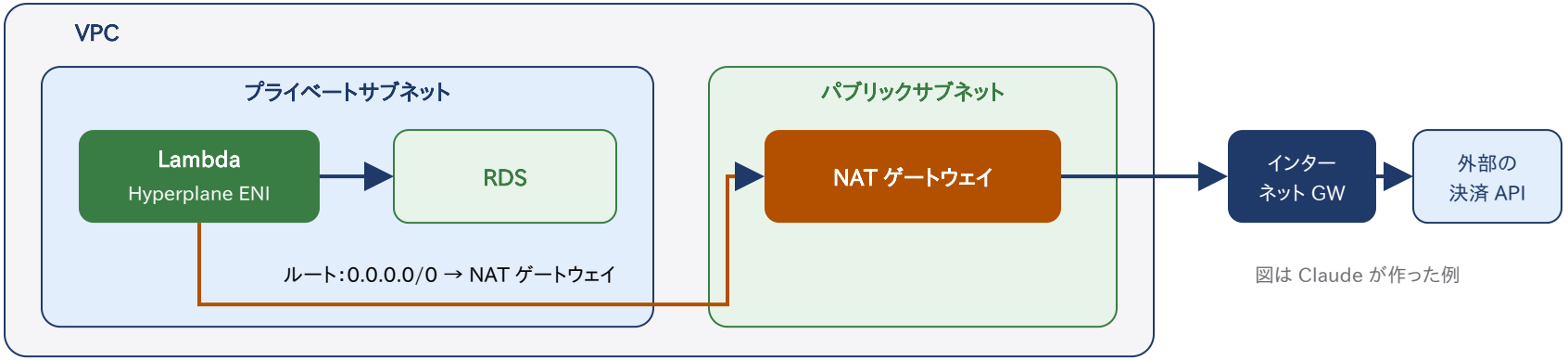
エイリアスで新しいバージョンを少しだけ試す



問題がなければ 100% に付け替える（Claude が作った例）

出典：レイヤーによる Lambda 依存関係の管理（「Lambda レイヤーは、補助的なコードやデータを含む .zip ファイルアーカイブです。」）／Lambda 関数のバージョンを管理する（「関数のバージョンを公開すると、そのコード、ランタイム、アーキテクチャ、メモリ、レイヤー、その他のほとんどの設定はイミュータブルになります。」）／Lambda 関数のエイリアスの作成（「Lambda のエイリアスとは、更新可能な関数のバージョンへのポインタです。」）

VPC の中のリソースを使う Lambda



図は Claude が作った例

ひっかけ：関数をパブリックサブネットに接続しても、パブリック IP は付かず、インターネットにも出られない。Lambda に Elastic IP は付けられない。

正しい形：関数はプライベートサブネット（2 つ以上の AZ）。インターネットへは NAT ゲートウェイ経由。AWS のサービスだけなら VPC エンドポイントでもよい。

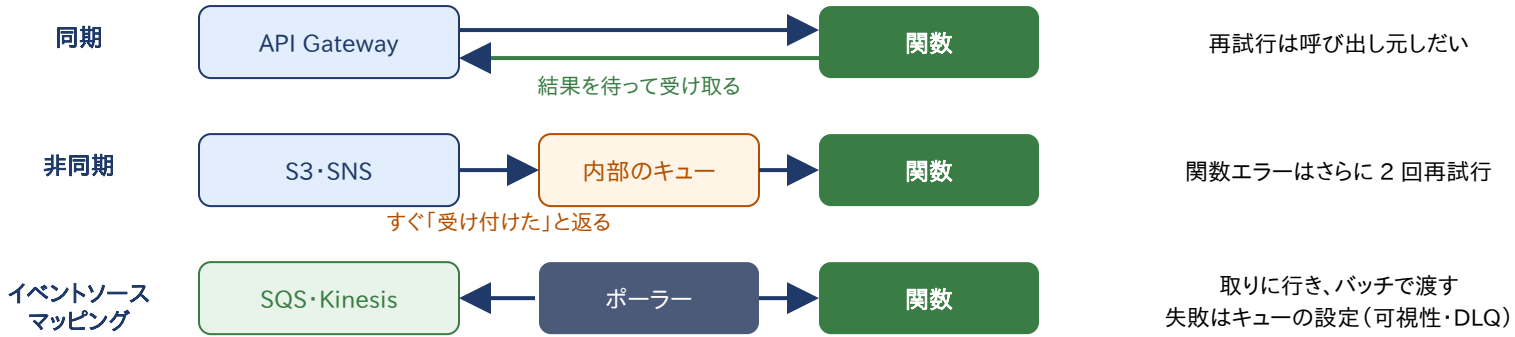
出典：Lambda 関数に Amazon VPC 内のリソースへのアクセスを許可する（「関数をパブリックサブネットに接続しても、インターネットアクセスやパブリック IP アドレスは提供されません。」）／VPC に接続された Lambda 関数にインターネットアクセスを有効にする（「プライベートサブネットは、NAT ゲートウェイ経由でインターネットにアクセスできます。」）

第2章

呼び出しとスケーリング

待つのか、任せるのか、取りに行くのか。失敗したときの動きと、同時に動く数の決め方。

3つの呼び出し方



見分け方：呼び出し元が結果を必要とする（API の応答）→ 同期。「置かれたら」「通知が来たら」→ 非同期。キューやストリーム → イベントソースマッピング。

SQS は押してこない：Lambda のポーラーがキューをポーリングする。関数の中で ReceiveMessage を回し続ける必要はない。

出典：Lambda 関数を同期的に呼び出す（「関数を同期的に呼び出すと、Lambda は関数を実行し、レスポンスを待ちます。」）／Lambda 関数を非同期的に呼び出す（「非同期呼び出しの場合、Lambda はリクエストをキューに入れ、追加情報のない成功のレスポンスを返します。」）／Lambda がストリームおよびキューベースのイベントソースからのレコードを処理する方法（「イベントソースマッピング内では、イベントポーラーと呼ばれるリソースが新しいメッセージをアクティブにポーリングし、関数を呼び出します。」）

非同期呼び出しが失敗したら



スロットリング（429）・システムエラー（500 番台）は、キューに戻して最大 6 時間試す

同じイベントが 2 回届くこともあるので、関数はべき等に作る

	送信先（Destinations）	デッドレターキュー（DLQ）
送るとき	成功時・失敗時を別々に設定できる	失敗して捨てられるときだけ
中身	呼び出しの詳細（レスポンス・エラー）を含む JSON の記録	**イベントの中身だけ**
送り先	標準 SQS・標準 SNS・Lambda・EventBridge・S3（失敗時のみ）	標準 SQS・標準 SNS

出典：Lambda がエラーを処理し、非同期呼び出しで再試行する方法（「その際、最初の 2 回の試行の間に 1 分間、2 回目と 3 回目の間に 2 分間の待機時間があります。」）／Lambda 非同期呼び出しのレコードのキャプチャ（「デッドレターキューの場合、Lambda はイベントのコンテンツのみを送信し、レスポンスの詳細は送信しません。」）

同時実行：取り合い・確保・温めておく

リージョンの同時実行 1,000 を全関数で分け合う（例）

決済 予約 300 ほかに取られない	残り 700 をほかの関数で取り合う 足りないとスロットリング

同時実行 ＝ 1 秒あたりのリクエスト数 × 平均の実行時間（秒）

例：200 件/秒 × 0.5 秒 ＝ 100（Claude が作った例）

上限にもなる：予約済み同時実行は下限であり上限。古い DB に同時に 50 本までしかつなげないなら 50 に絞る。

	何をする	料金
予約済み同時実行	枠を確保し、上限にもする	追加なし
プロビジョニングされた同時実行	初期化済みの環境を前もって用意（数十ミリ秒で応答）。バージョン・エイリアスに設定	追加あり
SnapStart	初期化済みのスナップショットから再開。用意は不要（Java・Python・.NET）	Java は追加なし。ほかはスナップショットの料金

1 つの関数は 10 秒ごとに最大 1,000 の実行環境ずつ増やせる。1,000 は引き上げを申請できる。

出典：Lambda 関数のスケーリングについて（「デフォルトで、アカウントの同時実行はリージョン内のすべての関数全体で 1,000 に制限されています。」）／関数に対する予約済み同時実行数の設定（「予約済同時実行数は、下限と上限の両方の役割を果たします。」）／Lambda SnapStart による起動パフォーマンスの向上（「アプリケーションに SnapStart で適切に対処できない厳格なコールドスタートレイテンシー要件がある場合は、プロビジョニングされた同時実行を使用します。」）

第3章

API Gateway

API の玄関。種類（REST・HTTP・WebSocket）と公開のしかたを選び、認可・スロットリング・キャッシュで守る。

REST API・HTTP API・WebSocket API

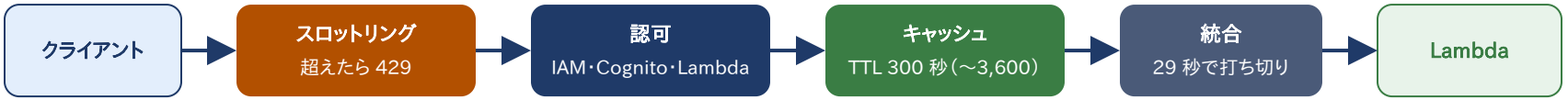
	REST API	HTTP API	WebSocket API
ひとことで	機能が多い	最小限の機能で低価格	双方向・ステートフル
REST だけの機能	API キーと使用量プラン・キャッシュ・AWS WAF・リクエストの検証・Canary	**なし**	—
認可	IAM・Cognito・Lambda オーソライザー・リソースポリシー	IAM・Lambda オーソライザー・JWT（Cognito・OIDC）	—
エンドポイント	エッジ最適化・リージョン・プライベート	**リージョンだけ**	—
向く場面	取引先に API キーを配る／VPC 内だけで使う	Lambda を呼ぶだけの安い API	チャット・リアルタイム通知

エンドポイントタイプ：エッジ最適化＝最寄りの CloudFront の POP 経由（世界中から）。リージョン＝同じリージョンから。プライベート＝インターフェイス VPC エンドポイント経由で VPC の中だけ。

ひっかけ：「HTTP API をエッジ最適化に」「HTTP API に API キー・キャッシュ・WAF」はできない。要件に出たら REST API。

出典：REST API と HTTP API のどちらかを選択する（「REST API は HTTP API よりも多くの機能をサポートしていますが、HTTP API は低価格で提供できるように最小限の機能で設計されています。」）／API Gateway での REST API の API エンドポイントタイプ（「API エンドポイントタイプは、API トラフィックの大部分の発信元となっている場所に応じて、エッジ最適化、リージョン別、またはプライベートとすることができます。」）

リクエストが通る順番：守る・速くする



アカウント単位：リージョンごと 10,000 リクエスト/秒（バースト 5,000）。ステージ・メソッド・使用量プランでさらに絞れる

順番は Claude が作った模式図（キャッシュは REST API だけ）

認可のしかた	使う場面
IAM 認可	AWS の認証情報で署名して呼ぶ（社内のシステム・ほかの AWS アカウント）
Cognito オーソライザー	Cognito のユーザープールでログインしたユーザーのトークンをそのまま検証
Lambda オーソライザー	独自のトークンやヘッダーを自分のコードで判断し、IAM ポリシーを返す
API キー	**識別と使用量の管理用。認証・認可には使わない**

出典：API Gateway での REST API のキャッシュ設定（「API キャッシュのデフォルトの TTL 値は 300 秒です。」）／API Gateway での REST API の設定および実行に関するクォータ（「Lambda、Lambda プロキシ、HTTP、HTTP プロキシ、AWS 統合など、すべての統合タイプで 50 ミリ秒〜29 秒。」）／API Gateway Lambda オーソライザーを使用する（「Lambda オーソライザーは、発信者の ID を入力として受け取り、IAM ポリシーを出力として返します。」）／API Gateway での REST API の使用量プランと API キー（「API キーを、API へのアクセスを制御するための認証または認可に使用しないでください。」）

第4章

Step Functions と Fargate

何ステップもの処理は Step Functions でつなぐ。15 分を超えるもの
や常に動くものはコンテナを Fargate で。

ステートマシンで注文処理を組む



状態	何をする	状態	何をする
Task	仕事をする（Lambda・AWS のサービス）	Map	配列の各要素に同じ処理を並列で
Choice	条件で次の状態を選ぶ	Wait	決めた秒数・日時まで待つ
Parallel	決まった枝を並列に動かす	Succeed・Fail	成功・失敗で終わる

出典：Step Functions とは（「Step Functions では、ステートマシンはワークフローと呼ばれます。」「ワークフローの各ステップはステートと呼ばれます。」）／Step Functions ワークフローでのエラー処理（「状態に Retry と Catch フィールドの両方がある場合、Step Functions は最初に適切な Retrier を使用します。」）

標準と Express、待ち方、Map の 2 つのモード

	標準	Express
最長	1 年	5 分
実行	ちょうど 1 回	非同期は少なくとも 1 回、同期は多くて 1 回
料金	状態遷移の回数	回数・時間・メモリ
向く処理	承認待ち・決済・監査	IoT・ストリーミングの大量の短い処理

Map 状態

	インライン	分散
並列	最大 40	最大 10,000（子ワークフロー）
向く処理	小さな配列	S3 の大量のファイル・CSV

サービス統合パターン	どこまで待つ
リクエストレスポンス	呼んで応答を受けたらすぐ次へ
ジョブの実行（.sync）	Batch・ECS のタスクなどのジョブが終わるまで
コールバック（.waitForTaskToken）	渡したタスクトークンが返ってくるまで（人の承認・外部システム）

Express ではできない：.sync・.waitForTaskToken・分散 Map は標準ワークフローだけ。長い待ちがある時点で標準。

出典：Step Functions でワークフロータイプを選択する（「Standard ワークフローは、長時間実行され (最長 1 年)、耐久性が高く、監査可能なワークフローに最適です。」）／Step Functions でサービス統合パターンを検出する（「ジョブの実行 (.sync) – サービスを呼び出し、ジョブが完了するまで Step Functions が待機するようにします。」）／Map 状態のワークフロー（「これにより、最大 10,000 件という高い並列性を持つの並列子ワークフロー実行が可能になります。」）

Lambda か Fargate か：どこで動かす

	Lambda	Fargate（ECS・EKS）	EC2
動かすもの	関数（.zip かコンテナイメージ）	コンテナ	何でも（OS から）
サーバー管理	不要	不要	必要（パッチ・台数）
1 回の時間	最大 15 分	上限なし（常に動くサービスも）	上限なし
起動のきっかけ	イベント・API	タスク・サービスとして起動	ずっと動く
安くする	メモリと時間を見直す	Fargate Spot（中断の 2 分前に警告）	リザーブド・スポット（#22）

Fargate の分離：各タスクはカーネル・CPU・メモリ・ENI をほかのタスクと共有しない。CPU とメモリを決めるだけで、インスタンスの種類選びやクラスターのスケールはいらない。

ステートレスに作る：Lambda も Fargate のタスクも、いつ増えて消えるか分からない。セッションやファイルは DynamoDB・ElastiCache・S3 に置けば、自由に台数を増やせる（水平スケーリング）。

出典：Amazon ECS の AWS Fargate 用のアーキテクト（「各Fargate タスクは、独自の分離境界を持ち、基盤となるカーネル、CPU リソース、メモリリソース、Elastic Network Interface を別のタスクと共有しません。」）／Fargate 用の Amazon ECS クラスター（「AWS がキャパシティを戻す必要がある場合、タスクは中断され、2 分間の警告が表示されます。」）

選び分け：この要件ならこれ

要件	答え
2 時間かかるバッチをサーバー管理なしで	**Fargate（Lambda は 15 分まで）**
Lambda の CPU が足りない	**メモリを増やす**
3 GB の依存ライブラリを Lambda で	**コンテナイメージ（10 GB まで）**
新しい版に 10% だけ流す	**エイリアスで振り分け**
VPC 内の Lambda から外部 API へ	**プライベートサブネット＋NAT ゲートウェイ**
非同期の失敗を詳細つきで残す	**失敗時の送信先**
重要な関数を取り合いから守る／DB への同時接続を絞る	**予約済み同時実行**

要件	答え
コールドスタートをなくす（追加料金 OK）	**プロビジョニングされた同時実行**
安くてシンプルな API（JWT）	**HTTP API**
API キー・使用量プラン・キャッシュ・WAF	**REST API**
VPC の中からだけ呼べる API	**プライベート REST API**
API の後ろの処理が 29 秒を超える	**受け付けだけ返して非同期に**
数日の承認待ちを含む業務	**標準ワークフロー ＋.waitForTaskToken**
S3 の 100 万ファイルを並列処理	**分散 Map**

出典：このPDFの各ページの資料

まとめ

Lambda の設定
メモリ=CPU。最大 15 分・/tmp は一時。依存はレイヤーかイメージ。エイリアスで切り替え

呼び出しと同時実行
同期・非同期（2 回再試行・送信先）・イベントソースマッピング。予約=確保と上限、プロビジョニング=温める

API Gateway
REST=多機能、HTTP=安い、WebSocket=双方向。キャッシュ 300 秒・統合 29 秒

Step Functions と Fargate
標準=長い・1 回、Express=短い・大量。15 分を超えるコンテナは Fargate

ひっかけ	正しくは
タイムアウトを 2 時間に	**最大 15 分**
パブリックサブネットで外へ	**NAT ゲートウェイ経由**
プロビジョニングで同時実行を絞る	**絞るのは予約済み**
API キーで認証	**識別用。認可は IAM など**
Express で数日待つ	**最長 5 分。標準を使う**

次の #08 では、ECS・EKS・ECR などのコンテナを学ぶ。

出典：このPDFの各ページの資料